

HW05 — AI-Assisted Performance Testing Report

Student information

Field	Value
Student	Nguyễn Hiền Tuấn Anh
Student ID	23127280
Tool	Apache JMeter 5.6.3
System under test	EShop backend on localhost:3000
Selected workflow	Scenario B — Coupon Purchase
Evidence date	2026-08-18

1. Executive summary

This assignment evaluates the complete Coupon Purchase workflow under normal load, staged stress, sudden traffic, and sustained activity. The accepted tests completed every business request with a 0.00% error rate. Their raw-JTL p95 response times ranged from 9 ms to 58 ms, well below the provisional 2,000 ms latency objective.

The application remained functional through the highest tested Stress stage of 30 VU, recovered after a Spike from 2 VU to 20 VU, and sustained approximately 26.4 samples/s during the ten-minute steady portion of the Soak test. Stress, Spike, and Soak produced short CPU warnings, but none met the emergency-stop rule of CPU above 95% for 60 consecutive seconds. No application breaking point was demonstrated within the tested range.

All four accepted official runs, hardware evidence, AI-assisted analysis, continuous-testing proposal, Agent Skill, and video mappings were reviewed and accepted by the student. Superseded or invalid runs are excluded from the reported performance results.

2. Scope and business workflow

Scenario B models one end-to-end purchase with a fixed coupon:

- authenticate through Login;
- search for a product and retrieve its details;
- add the product to the cart and verify the cart;
- apply the fixed coupon;
- checkout using the calculated final amount;
- record coupon usage; and
- verify the new order in Order History.

The workflow combines authentication, read-heavy catalog access, stateful cart operations, coupon validation, checkout writes, and a growing order-history query. Test data stays outside the JMX plans in CSV files and JMeter properties. Each measured run used CLI mode; GUI listeners were used only after execution to inspect the raw JTL.

3. Environment and acceptance rules

The official runs used the student-verified host TUANANH, an Acer Nitro AN515-57 with an Intel Core i5-11400H, 24 GB RAM, Windows 11, Temurin JDK 17 for JMeter, and JMeter 5.6.3. Calibration established a stable load level of L = 10 virtual users.

The report applies these provisional acceptance rules:

- p95 response time below 2,000 ms ;
- error rate below 1% ;
- CPU above 90% produces a warning;
- CPU above 95% for 60 consecutive seconds triggers an emergency stop; and
- RAM is judged by its start, peak, and final trend rather than by one isolated reading.

Each official session started from reset test data and a passing nine-request preflight. JMeter used a unique Run ID and refused to overwrite an existing JTL or non-empty HTML report directory. Resource counters sampled CPU and physical RAM once per second.

4. Test design

Profile	Purpose	Workload
Load	Measure normal behavior at calibrated concurrency	10 VU , ramp 120 s , hold 300 s

Profile	Purpose	Workload
Stress	Search for capacity degradation or failure	stages of 10 , 15 , 20 , and 30 vU ; 60 s ramp and 120 s duration per stage
Spike	Measure sudden-load response and recovery	2 vU baseline, ramp to 20 vU in 10 s , hold 60 s , ramp down in 10 s , recover at 2 vU for 120 s
Soak	Detect latency, throughput, error, or memory drift	7 vU , ramp 60 s , sustain 10 min

Load uses the calibrated level directly. Soak uses `floor(0.7L) = 7 vU` . Stress increases concurrency in controlled stages, while Spike changes concurrency abruptly and then returns to baseline.

5. Official results

The following table reports nearest-rank percentiles over all rows in each accepted raw JTL. Stress and Spike include both endpoint samples and parent end-to-end transaction samples.

Profile and accepted Run ID	Raw samples	Throughput	Raw p95	Raw p99	Errors	Peak CPU	Peak RAM	Result
Load 23127280_Load_Official_20260818_002	7,120	16.983 samples/s	9 ms	12 ms	0.00%	89.54%	79.17%	Pass latency and error objectives
Stress 23127280_Stress_Official_20260818_001	7,607	15.852 samples/s	35 ms	64 ms	0.00%	99.87%	78.67%	No application breaking point through 30 vU; transient resource warning
Spike 23127280_Spike_Official_20260818_002	36,598	140.984 samples/s	58 ms	151 ms	0.00%	97.44%	78.85%	Pass latency, errors, and recovery; transient CPU warning
Soak 23127280_Soak_Official_20260818_002	16,759	25.411 samples/s including ramp	9 ms	11 ms	0.00%	97.20%	72.44%	Pass latency, errors, and stability; transient CPU warning

5.1 Load interpretation

Load produced 7,120 successful request samples over approximately seven minutes. The 9 ms raw p95 and 0.00% error rate passed the acceptance objectives. CPU and RAM remained below the emergency thresholds. This run is the normal-load reference for the current machine and dataset.

5.2 Stress interpretation

Stress completed all stages through 30 vU without a failed sample. Raw p95 remained 35 ms ; therefore, the run did not demonstrate an application breaking point. CPU briefly reached 99.87% , but the high reading lasted one second rather than the required 60 consecutive seconds. The result is a resource-capacity warning, not a functional failure. The raw JTL contains 799 parent E2E rows and 6,808 endpoint rows. Its total of 7,607 differs from the HTML total of 6,883 , so the two reports use different sample populations. This report keeps the raw all-row metrics and labels their scope explicitly.

5.3 Spike interpretation

Spike _002 replaced an invalid first attempt whose coupon data was exhausted. The corrected run processed 36,598 samples with no failures. Its raw p95 was 58 ms , and the workload returned to the 2 vU recovery level without residual errors or latency degradation. CPU exceeded 95% for two seconds, which remains below the emergency-stop duration. The HTML dashboard reports 32,939 samples and p95 18 ms ; the raw JTL also contains 3,660 parent E2E rows with a higher latency distribution. Both values describe different populations. The official conclusion therefore uses the raw all-row p95 and records the dashboard value only as supporting evidence.

5.4 Soak interpretation

Soak sustained 7 vU for ten minutes and completed 16,759 samples without errors. Throughput remained nearly flat: 26.420 samples/s in the first sustained half and 26.397 samples/s in the last. Raw p95 improved from 9 ms overall to 8 ms in the last half. Physical RAM peaked at 72.44% and finished at 60.74% , providing no evidence of progressive memory growth during this run.

6. Cross-test findings

The accepted evidence supports four conclusions:

1. **Latency remained low.** Every raw p95 stayed far below the provisional 2,000 ms objective.
2. **Business correctness held.** All accepted runs recorded 0.00% errors across the full Coupon Purchase workflow.
3. **Recovery was successful.** The corrected Spike returned to baseline load without a failure cascade.
4. **Host CPU is the first visible constraint.** Short CPU peaks appeared under Stress, Spike, and Soak, but application latency and errors did not show a breaking point.

The results apply only to this local machine, dataset, JMeter version, and workload model. They do not establish production capacity or a universal concurrency limit.

7. AI-assisted analysis and human correction

AI helped compare the four profiles, propose thresholds, and identify optimization candidates. Human review found that the first analysis matched arithmetic differences to the wrong sampler and copied a visible dashboard percentile without identifying its population.

Three corrections shaped the final report:

- Stress has `799` parent transaction rows, not `724` ; `724` is the count of the final Order History sampler.
- Spike raw p95 is `58 ms` , while HTML p95 is `18 ms` ; both require an explicit population label.
- A brief CPU peak above `95%` is a warning, not proof of an application breaking point under the 60-second stop rule.

The review accepted two practical optimization directions: test composite indexes for order and coupon-usage lookups, and paginate Order History. SQLite WAL and a bounded busy timeout remain experiments rather than proven fixes. The review rejected a generic database connection pool and immediate Node clustering because the current service uses one local SQLite file and keeps cart state in process memory.

The main lesson is methodological: every metric must record its source, sample population, percentile method, and compatible baseline. AI can generate hypotheses, but raw evidence and source-code constraints decide the conclusion.

8. Continuous performance testing proposal

The accepted Task 3 proposal uses path-based triggers and four execution tiers:

Trigger	Check
Relevant pull request	1-VU preflight followed by a short Load regression
Nightly	Full accepted Load profile at <code>L = 10</code>
Weekly	Full Load plus the ten-minute Soak profile
Release candidate with approval	Stress or Spike in an isolated non-production environment

Documentation-only and unrelated frontend changes skip JMeter but record the reason. Performance-sensitive backend, database, test-data, JMX, and runner changes trigger the appropriate tier.

The proposed gate fails on error rate at least `1%` or comparable p95 at least `2,000 ms` . It flags a relative regression only when p95 rises by both more than `20%` and more than `10 ms` against a compatible baseline. A relative breach must repeat on the same commit and runner before the pipeline fails. Baselines require the same workload, hardware class, dataset, JMeter version, sample population, and percentile method.

This design reduces false alarms and execution cost. It keeps short feedback on pull requests, moves representative tests to schedules, and restricts disruptive Stress and Spike profiles to approved non-production runs. It is a proposal; no CI workflow is claimed as implemented.

9. Reusable Agent Skill

The submitted, student-reviewed Agent Skill packages the guarded performance workflow for future endpoint groups. It validates JMX structure and CSV variables, requires a passing preflight, runs measured tests in CLI mode, preserves immutable evidence, and prevents automation from assigning human acceptance. All official evidence included in this submission is `HUMAN_VERIFIED` .

Scenario B remains the default contract. A JSON contract can define another endpoint group's sampler order, correlations, CSV fields, filename rules, listeners, and forbidden literals without changing validator code. The demonstration covered plan validation, guarded execution, JTL summarization, evidence states, and reusable contract support.

10. Evidence integrity, accepted limitations, and final verdict

Three invalid or superseded sessions were excluded from official metrics: contaminated Load `_001` , coupon-exhausted Spike `_001` , and agent-operated Soak `_001` . Each was replaced by a student-operated, human-verified run. This preserves the distinction between product behavior and failures caused by evidence handling, test data, or execution ownership. The student reviewed and accepted one historical traceability limitation: the Stress manifest contains stale hashes for the canonical Stress JMX and `users.csv` after those working files changed, and no run-local copies of those two inputs were retained. The Stress raw JTL, HTML report, result summary, resource counters, screenshots, and accepted verdict remain intact. The historical manifest remains unchanged to preserve evidence integrity.

Within the tested environment, Scenario B passes the latency and error objectives for Load, Stress, corrected Spike, and Soak. The application showed no breaking point through `30 VU` , recovered from a `20 VU` spike, and remained stable for the ten-minute Soak. The final verdict is **PASS with transient host-resource warnings and the documented Stress input-traceability limitation**.

References

Main analysis and review

- [Task 2 — AI Analysis and Misinterpretation Hunt](#)
- [AI Critique](#)
- [Task 3 — Continuous Performance Testing Proposal](#)
- [AI Audit](#)

Accepted plans and raw results

- Load: [JMX](#), [raw JTL](#), [HTML report](#)
- Stress: [JMX](#), [raw JTL](#), [HTML report](#)
- Spike: [JMX](#), [raw JTL](#), [HTML report](#)
- Soak: [JMX](#), [raw JTL](#), [HTML report](#)

Supporting evidence and reusable workflow

- [Hardware evidence](#)
- [Load tool and resource-monitor evidence](#)
- [Video evidence and accepted Run IDs](#)
- [Reusable performance-testing Agent Skill](#)
- [Git commit history](#)